React에서 useState, useMemo, useRef는 모두 Hook(훅) 이라고 부릅니다.

React 컴포넌트에서 데이터를 관리하거나, 계산을 효율적으로 처리하거나, 특정 요소에 접근할 때 사용하는 기능입니다.



처음 React를 배우는 학생이라면 다음과 같이 이해하면 쉽습니다.

Hook	쉽게 설명하면	주요 역할
`useState`	값을 기억하고 화면도 바꾸는 메모장	상태 관리
`useMemo`	계산 결과를 기억하는 계산기	불필요한 재계산 방지
`useRef`	값을 보관하거나 HTML 요소를 가리키는 보관함	DOM 접근, 값 유지

1. useState — 값이 바뀌면 화면도 바뀐다

가장 먼저 배워야 하는 Hook입니다.

예를 들어 카페에서 주문한 커피의 수량을 표시하는 화면을 만든다고 생각해 보겠습니다.

실습 예제 — 커피 주문 수량



총 주문 금액: **4,500원**

JavaScript

```javascript
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>커피 주문: {count}잔</h2>

      <button onClick={() => setCount(count + 1)}>
        추가
      </button>
    </div>
  );
}
```

2. useMemo — 계산한 결과를 기억한다

`useMemo`는 **계산 결과를 저장해 두었다가 필요한 경우에만 다시 계산**합니다.

예를 들어 커피 100개의 가격을 계산해야 한다고 생각해 보겠습니다. 주문 수량이 바뀌지 않았다면 매번 다시 계산할 필요가 없습니다.

JavaScript

```
  const total = useMemo(() => {
    console.log("금액 계산!");
    return count * 4500;
  }, [count]);

  return (
    <div>
      <h2>총 금액: {total}원</h2>

      <button onClick={() => setCount(count + 1)}>
        커피 추가
      </button>

      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="고객 이름"
      />
    </div>
  );
}
```

3. useRef — 값을 기억하지만 화면을 다시 그리지 않는다

`useRef`는 두 가지 용도로 많이 사용합니다.

첫 번째는 **HTML 요소에 직접 접근**하는 것입니다.

JavaScript

```
import { useRef } from "react";

function App() {
  const inputRef = useRef(null);

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <div>
      <input ref={inputRef} placeholder="이름 입력" />

      <button onClick={focusInput}>
        입력창 선택
      </button>
    </div>
  );
}
```

아래 예제에서 버튼을 누르면 입력창에 커서가 이동합니다.

실습 예제 — 입력창 선택

이름을 입력하세요

입력창에 커서 이동

React에서는 `inputRef.current.focus()`로 입력창을 선택할 수 있습니다.



우리 반 카페

다크 모드

메뉴 선택

메뉴 검색

전체 | 음료 | 디저트



아메리카노
3,000원
담기



카페라테
3,500원
담기



쿠키
2,000원
담기



치즈케이크
5,000원
담기

장바구니

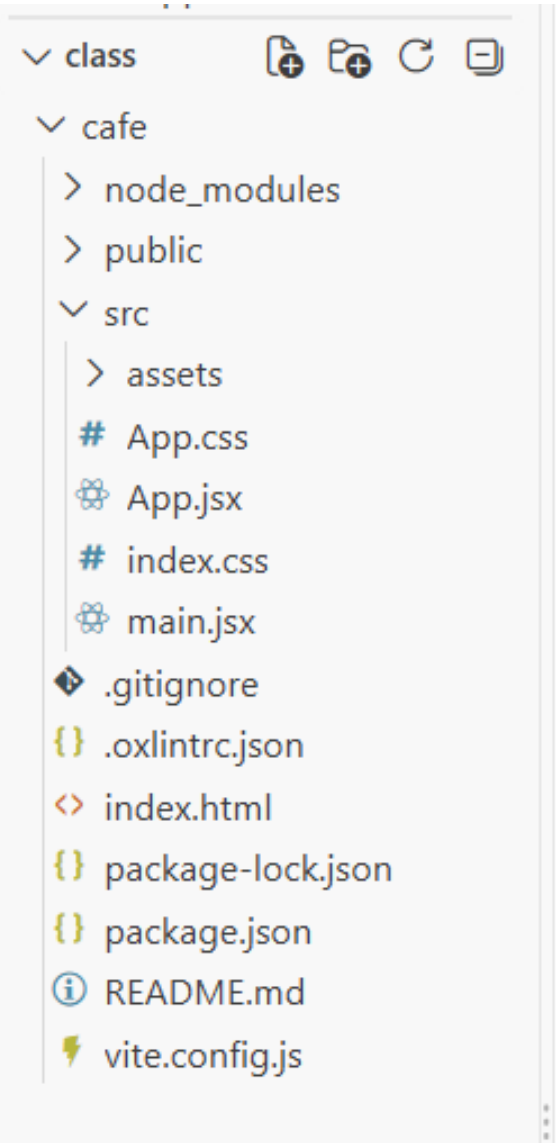


장바구니가
비어 있어요.

주문 결과



주문 후 결과를
확인하세요.



App.jsx 내용을 모두 지우고 아래 코드를 넣으세요

```
import "./App.css";

export default function App() {
  return (
    <div className="app">
      <h1>우리 반 카페</h1>
      <p>화면 구성 실습을 시작합니다.</p>
    </div>
  );
}
```

우선 VS Code에서 **터미널 → 새 터미널**을 열고, 아래 명령을 한 줄씩 입력하세요.

```powershell
cd C:\Users\ej070\class\cafe
```

```powershell
npm run dev
```

① cafe/src/App.jsx 내용을 모두 아래 코드로 바꾸세요.

```
import "./App.css";

export default function App() {
  return (
    <div className="app">
      <h1>우리 반 카페</h1>
      <p>원하는 메뉴를 골라 보세요.</p>

      <main className="layout">
        <section className="panel">
          <h2>메뉴 선택</h2>
          <p>여기에 메뉴가 표시됩니다.</p>
        </section>

        <section className="panel">
          <h2>장바구니</h2>
          <p>장바구니가 비어 있어요.</p>
        </section>

        <section className="panel">
          <h2>주문 결과</h2>
          <p>주문 후 결과를 확인하세요.</p>
        </section>
      </main>
    </div>
  );
}
```



② 같은 src 폴더의 App.css 내용을 모두 아래 코드로 바꾸세요.

```
.app {
  max-width: 1200px;
  margin: 0 auto;
  padding: 30px;
  color: #163043;
}

.layout {
  display: grid;
  grid-template-columns: 1.4fr 1fr 1fr;
  gap: 20px;
  margin-top: 25px;
}

.panel {
  padding: 24px;
  background: white;
  border: 1px solid #ddd;
  border-radius: 16px;
}

.panel h2 {
  margin-top: 0;
}

@media (max-width: 800px) {
  .layout {
    grid-template-columns: 1fr;
  }
}
```

③ src/index.css 내용도 모두 아래 코드로 바꾸세요.

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: #f7f3eb;
  font-family: Arial, "맑은 고딕", sans-serif;
}
```



메뉴 카드 한 개 넣기 :

① src/App.jsx에서 아래 줄을 찾으세요.

```
<p>여기에 메뉴가 표시됩니다.</p>
```

이 한 줄을 지우고 아래 코드로 바꾸세요.

```
<div className="menu-card">
  <h3>아메리카노</h3>
  <p>3,000원</p>
  <button>담기</button>
</div>
```



② src/App.css 맨 아래에 다음 코드를 추가하세요.
기존 CSS는 그대로 두세요.

```
.menu-card {
  padding: 20px;
  border: 1px solid #ddd;
  border-radius: 12px;
  text-align: center;
}

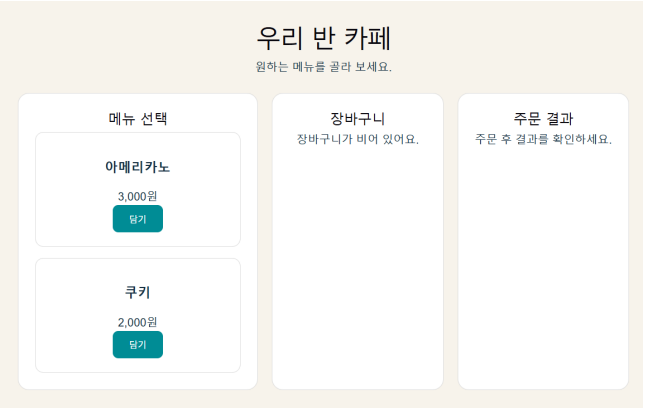
.menu-card button {
  padding: 10px 24px;
  background: #008c95;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}
```



③ 두 파일을 저장하세요.
메뉴 선택 영역에 **아메리카노·3,000원·담기 버튼**이 보이면 성공입니다. 지금은 버튼 모양까지 만들고, 클릭 동작은 나중에 연결합니다.

이 카드를 **여러 번 사용할 수 있는 컴포넌트**

① src/App.jsx 내용을 모두 아래 코드로 바꾸세요.




```
import "./App.css";

// 메뉴 카드 컴포넌트
function MenuCard({ name, price }) {
  return (
    <div className="menu-card">
      <h3>{name}</h3>
      <p>{price.toLocaleString("ko-KR")}원</p>
      <button>담기</button>
    </div>
  );
}

// 전체 화면
export default function App() {
  return (
    <div className="app">
      <h1>우리 반 카페</h1>
      <p>원하는 메뉴를 골라 보세요.</p>

      <main className="layout">
        <section className="panel">
          <h2>메뉴 선택</h2>
          <MenuCard name="아메리카노" price={3000} />
          <MenuCard name="쿠키" price={2000} />
        </section>

        <section className="panel">
          <h2>장바구니</h2>
          <p>장바구니가 비어 있어요.</p>
        </section>

        <section className="panel">
          <h2>주문 결과</h2>
          <p>주문 후 결과를 확인하세요.</p>
        </section>
      </main>
    </div>
  );
}
```

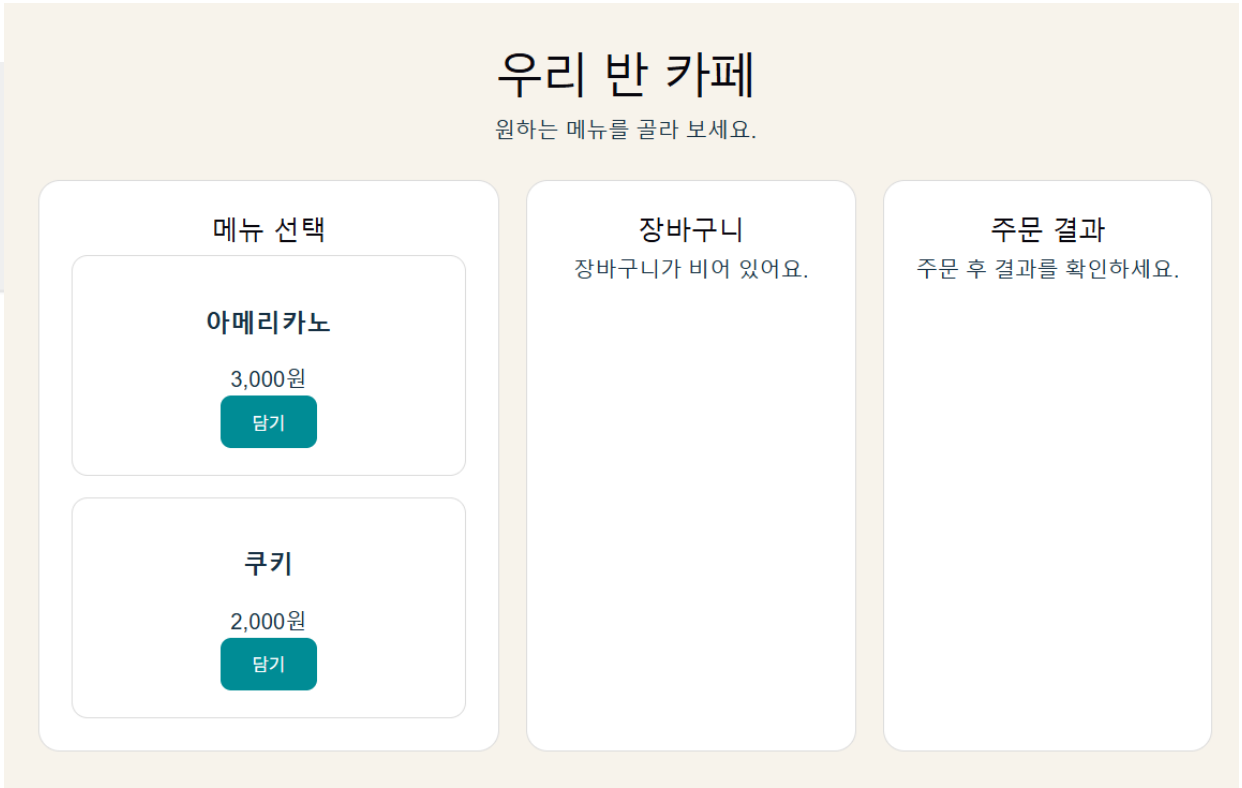
② App.css 맨 아래에 다음 코드를 추가하세요.

```
.menu-card + .menu-card {
  margin-top: 16px;
}
```

③ 저장하고 확인하세요.
아메리카노 3,000원과 쿠키 2,000원, 카드 두 개가 세로로 보이면 성공입니다.
이번 단계에서 알아둘 것은 **Props**입니다.

JavaScript

```
<MenuCard name="쿠키" price={2000} />
```



메뉴를 배열에 저장하고, map()으로 **6개를 자동 표시**

```
import "./App.css";
// 메뉴 데이터
const menus = [
  { id: 1, name: "아메리카노", price: 3000 },
  { id: 2, name: "카페라테", price: 3500 },
  { id: 3, name: "초코라테", price: 4000 },
  { id: 4, name: "레모네이드", price: 4000 },
  { id: 5, name: "쿠키", price: 2000 },
  { id: 6, name: "치즈케이크", price: 5000 },
];
// 메뉴 카드
function MenuCard({ name, price }) {
  return (
    <div className="menu-card">
      <h3>{name}</h3>
      <p>{price.toLocaleString("ko-KR")}원</p>
      <button>담기</button>
    </div>
  );
}
// 전체 화면
export default function App() {
  return (
    <div className="app">
      <h1>우리 반 카페</h1>
      <p>원하는 메뉴를 골라 보세요.</p>
      <main className="layout">
        <section className="panel">
          <h2>메뉴 선택</h2>

          <div className="menu-grid">
            {menus.map((menu) => (
              <MenuCard
                key={menu.id}
                name={menu.name}
                price={menu.price}
              />
            ))}
          </div>
        </section>

        <section className="panel">
          <h2>장바구니</h2>
          <p>장바구니가 비어 있어요.</p>
        </section>

        <section className="panel">
          <h2>주문 결과</h2>
          <p>주문 후 결과를 확인하세요.</p>
        </section>
      </main>
    </div>
  );
}
```

② src/App.css에서 아래 코드를 찾아 삭제하세요.

```
.menu-card + .menu-card {
  margin-top: 16px;
}
```

③ App.css 맨 아래에 아래 코드를 추가하세요.

```
.menu-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 14px;
}

.menu-card {
  min-width: 0;
  padding: 16px;
}

.menu-card h3 {
  font-size: 17px;
}
```

④ 두 파일을 저장하세요.

메뉴 카드가 **가로 2개씩, 총 6개** 보이면 성공입니다.

코드	역할
`menus`	메뉴 6개의 정보를 저장한 배열
`menus.map()`	메뉴 하나마다 카드 하나를 만드는 코드
`key={menu.id}`	React가 각 카드를 구분하도록 주는 식별값



‘담기’를 누르면 장바구니에 메뉴가 표시

① src/App.jsx 내용을 모두 아래 코드로 바꾸세요.

```
import { useState } from "react";
import "./App.css";

const menus = [
  { id: 1, name: "아메리카노", price: 3000 },
  { id: 2, name: "카페라테", price: 3500 },
  { id: 3, name: "초코라테", price: 4000 },
  { id: 4, name: "레모네이드", price: 4000 },
  { id: 5, name: "쿠키", price: 2000 },
  { id: 6, name: "치즈케이크", price: 5000 },
];

// onAdd는 담기 버튼을 클릭했을 때 실행할 함수입니다.
function MenuCard({ name, price, onAdd }) {
  return (
    <div className="menu-card">
      <h3>{name}</h3>
      <p>{price.toLocaleString("ko-KR")}원</p>
      <button onClick={onAdd}>담기</button>
    </div>
  );
}

export default function App() {
  // 장바구니의 처음 값은 빈 배열입니다.
  const [cart, setCart] = useState([]);

  // 기존 장바구니에 선택한 메뉴를 추가합니다.
  function addToCart(menu) {
    setCart((previous) => [...previous, menu]);
  }

  return (
    <div className="app">
      <h1>우리 반 카페</h1>
      <p>원하는 메뉴를 골라 보세요.</p>

      <main className="layout">
        <section className="panel">
          <h2>메뉴 선택</h2>

          <div className="menu-grid">
            {menus.map((menu) => (
```

‘담기’를 누르면 장바구니에 메뉴가 표시

① src/App.jsx 내용을 모두 아래 코드로 바꾸세요.

```
          <MenuCard
            key={menu.id}
            name={menu.name}
            price={menu.price}
            onAdd={() => addToCart(menu)}
          />
        ))}
      </div>
    </section>

    <section className="panel">
      <h2>장바구니</h2>

      {cart.length === 0 ? (
        <p>장바구니가 비어 있어요.</p>
      ) : (
        <div>
          {cart.map((item, index) => (
            <p key={index}>
              {item.name}
              {" "}
              {item.price.toLocaleString("ko-KR")}원
            </p>
          ))}
        </div>
      )}
    </section>

    <section className="panel">
      <h2>주문 결과</h2>
      <p>주문 후 결과를 확인하세요.</p>
    </section>
  </main>
</div>
  );
}
```

② App.css 맨 아래에 다음 코드를 추가하세요.
화면에서 첫 번째 카드와 두 번째 카드의 높이가 어긋나 있어, 남아 있는 카드 위쪽 여백을 없애겠습니다.

```
.menu-grid > .menu-card {
  margin-top: 0;
}
```

③ 저장한 뒤 다음 순서대로 눌러보세요.

실행	장바구니에 표시될 내용
아메리카노 '담기' 클릭	아메리카노 3,000원
쿠키 '담기' 클릭	그 아래에 쿠키 2,000원 추가
아메리카노 '담기' 다시 클릭	아메리카노 한 줄 추가



장바구니에서 –, +, 삭제 버튼으로 직접 조절하는 기능

① App.jsx의 App 함수 안에서, return (바로 위에 아래 함수 두 개를 추가하세요.

```
// 수량 변경: 0개가 되면 장바구니에서 제외
function changeQuantity(id, amount) {
  setCart((previous) =>
    previous
      .map((item) =>
        item.id === id
          ? { ...item, quantity: item.quantity + amount }
          : item
      )
      .filter((item) => item.quantity > 0)
  );
}

// 선택한 메뉴 삭제
function removeFromCart(id) {
  setCart((previous) =>
    previous.filter((item) => item.id !== id)
  );
}
```



② 장바구니의 {cart.map(...)} 부분 전체를 아래 코드로 교체하세요.
바깥의 cart.length === 0 조건문은 유지하세요.

```
{cart.map((item) => (
  <div key={item.id} className="cart-item">
    <h3>{item.name}</h3>

    <div className="quantity-controls">
      <button onClick={() => changeQuantity(item.id, -1)}>
        −
      </button>

      <span>{item.quantity}개</span>

      <button onClick={() => changeQuantity(item.id, 1)}>
        +
      </button>

      <button onClick={() => removeFromCart(item.id)}>
        삭제
      </button>
```

③ App.css 맨 아래에 추가하세요.

```
.quantity-controls {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}

.quantity-controls button {
  padding: 6px 12px;
  background: #edf3f4;
  color: #163043;
  border: 1px solid #ccd4d6;
  border-radius: 6px;
  cursor: pointer;
}
```

저장한 뒤 아래 동작을 확인하세요.



조작	예상 결과
+ 클릭	수량과 금액 증가
- 클릭	수량과 금액 감소
1개에서 - 클릭	해당 메뉴 삭제
삭제 클릭	해당 메뉴 전체 삭제

장바구니 전체 금액을 계산

① App.jsx 맨 위의 import를 바꾸세요.

```
import { useState, useMemo } from "react";
import "./App.css";
```

② App 함수 안에서 아래 상태 선언을 찾으세요.

```
const [cart, setCart] = useState([]);
```

그 바로 아래에 추가하세요.

```
const totalPrice = useMemo(() => {
  return cart.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );
}, [cart]);
```

③ 장바구니 제목을 찾으세요.

```
<h2>장바구니</h2>
```

그 바로 아래에 추가하세요.

```
<div className="cart-total">
  총 주문 금액
  <strong>{totalPrice.toLocaleString("ko-KR")}원</strong>
</div>
```

④ App.css 맨 아래에 추가하세요.

```
.cart-total {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 10px;
  padding: 16px;
  margin-bottom: 16px;
  background: #e7f5f4;
  color: #007b83;
  border-radius: 10px;
  font-size: 16px;
}

.cart-total strong {
  font-size: 22px;
}
```

코드	설명
`reduce()`	각 메뉴의 가격 × 수량을 모두 더합니다.
`0`	합계를 0부터 시작합니다.
`[cart]`	장바구니가 바뀌면 합계를 다시 계산하도록 합니다.
`useMemo()`	계산 결과를 저장해 재사용합니다.

우리 반 카페

원하는 메뉴를 골라 보세요.

메뉴 선택

아메리카노

3,000원

담기

카페라테

3,500원

담기

초코라테

4,000원

담기

레모네이드

4,000원

담기

쿠키

2,000원

담기

치즈케이크

5,000원

담기

장바구니

총 주문 금액 **16,000원**

아메리카노

− 3개 + 삭제

9,000원

카페라테

− 2개 + 삭제

7,000원

주문 결과

주문 후 결과를 확인하세요.

‘주문하기’를 누르면 오른쪽에 주문 결과를 표시하는 기능



① App.jsx의 App 함수 안에서 아래 줄을 찾으세요.

```
const [cart, setCart] = useState([]);
```

그 바로 아래에 추가하세요.

```
const [order, setOrder] = useState(null);
```

order는 완료한 주문을 저장합니다. 처음에는 주문이 없으므로 null입니다.

② App 함수 안의 return (바로 위에 추가하세요.

```
function placeOrder() {
  if (cart.length === 0) {
    alert("메뉴를 먼저 담아 주세요.");
    return;
  }

  // 현재 메뉴와 금액을 주문 결과에 저장합니다.
  setOrder({
    items: cart.map((item) => ({ ...item })),
    total: totalPrice,
  });

  // 주문 후 장바구니를 비웁니다.
  setCart([]);
}
```

③ 장바구니의 총금액 표시 부분 바로 아래에 버튼을 추가하세요.

```
<button className="order-button" onClick={placeOrder}>
  주문하기
</button>
```

④ 기존 주문 결과 영역 전체를 아래 코드로 교체하세요.

```
<section className="panel">
  <h2>주문 결과</h2>
  {order === null ? (
    <p>주문 후 결과를 확인하세요.</p>
  ) : (
    <div className="order-result">
      <h3>주문 완료!</h3>
      {order.items.map((item) => (
        <p key={item.id}>
          {item.name} {item.quantity}개
        </p>
      ))}
      <hr />
      <strong>
        주문 금액: {order.total.toLocaleString("ko-KR")}원
      </strong>
    </div>
  )}
</section>
```

⑤ App.css 맨 아래에 추가하세요.

```
.order-button {
  width: 100%;
  padding: 14px;
  margin-bottom: 20px;
  background: #008c95;
  color: white;
  border: none;
  border-radius: 10px;
  font-size: 18px;
  cursor: pointer;
}

.order-result {
  padding: 16px;
  background: #e7f5f4;
  border-radius: 10px;
}
```



주문자 이름을 입력하고, 이름이 없으면 입력창으로 커서를 이동하는 기능



① App.jsx 맨 위의 React import 한 줄을 교체하세요.

```
import { useState, useMemo, useRef } from "react";
import "./App.css";
```

② App 함수 안에서 다음 코드 아래에 추가하세요.

찾을 코드:

```
const [order, setOrder] = useState(null);
```

추가할 코드:

```
const [customerName, setCustomerName] = useState("");
const nameInputRef = useRef(null);
```

③ 기존 placeOrder 함수 전체를 아래 코드로 교체하세요.

```
function placeOrder() {
  if (cart.length === 0) {
    alert("메뉴를 먼저 담아 주세요.");
    return;
  }
  if (customerName.trim() === "") {
    alert("주문자 이름을 입력해 주세요.");
    nameInputRef.current?.focus();
    return;
  }
  setOrder({
    customerName: customerName.trim(),
    items: cart.map((item) => ({ ...item })),
    total: totalPrice,
  });
  setCart([]);
  setCustomerName("");
}
```

④ 장바구니의 '주문하기' 버튼 바로 위에 추가하세요.

```
<div className="customer-field">
  <label htmlFor="customer-name">주문자 이름</label>

  <input
    id="customer-name"
    ref={nameInputRef}
    value={customerName}
    onChange={(event) => setCustomerName(event.target.value)}
    placeholder="이름을 입력하세요"
  />
</div>
```

⑤ 주문 결과의 다음 줄을 찾으세요.

```
<h3>주문 완료!</h3>
```

그 바로 아래에 추가하세요.

```
<p>
  <strong>{order.customerName}</strong> 님의 주문입니다.
</p>
```

⑥ App.css 맨 아래에 추가하세요.

```
.customer-field {
  margin: 16px 0;
  text-align: left;
}

.customer-field label {
  display: block;
  margin-bottom: 8px;
  font-weight: bold;
}

.customer-field input {
  width: 100%;
  padding: 12px;
  border: 1px solid #ccd4d6;
  border-radius: 8px;
  font: inherit;
}

.customer-field input:focus {
  outline: 2px solid #008c95;
  outline-offset: 2px;
}
```

저장한 뒤 다음 순서로 확인하세요.

실행	예상 결과
메뉴를 담고, 이름 없이 주문	안내창을 닫으면 이름 입력창에 커서 표시
이름을 입력하고 주문	주문 결과에 이름·메뉴·금액 표시
주문 완료 후	장바구니와 이름 입력창 초기화